

AI-Based Ecological Predictive Cruise Control for Connected Electric Vehicles Using LSTM-Enhanced PID and Model Predictive Control

Riyan Wankhede

Department of Computer Science and Engineering
Vellore Institute of Technology, Andhra Pradesh
(VIT-AP)
riyanwankhede@gmail.com

Nishita

Department of Computer Science and Engineering
Vellore Institute of Technology, Andhra Pradesh
(VIT-AP)
nishitay1652@gmail.com

Abstract - This paper evaluates whether a stacked long short-term memory (LSTM) forecast of the traffic speed ahead makes an ecological predictive cruise controller for an electric vehicle (EV) more energy-efficient. A 2,500 kg EV model with a single-speed reduction gear, motor torque and power limits, and blended regenerative and friction braking follows the EPA UDDS (city), HWFET (highway) and US06 (aggressive) drive cycles, none of which is used for training or tuning. Four controllers are compared: bang-bang, PID, PID with deceleration feedforward (LSTM-PID) and a receding-horizon model predictive controller (LSTM-MPC). Each forecast-based controller is run with four forecasts: persistence, constant-acceleration extrapolation, the LSTM, and an oracle that knows the true future. Because energy can always be saved by tracking the reference more loosely, energy is compared at equal tracking error by sweeping the MPC energy weight. With a perfect 2 s preview, the MPC matches the PID's tracking with 2.6%, 0.7% and 4.3% less energy on the city, highway and aggressive cycles. The LSTM recovers little of this: over five training seeds the saving is $1.0\% \pm 0.8$, $0.0\% \pm 0.6$ and $-0.9\% \pm 1.7$. Its forecasts are about as accurate as constant-acceleration extrapolation, and their step-to-step variation raises the MPC's jerk $2.5\text{--}2.8\times$ above the oracle MPC's. The deceleration feedforward does not change PID energy ($\leq 0.4\%$).

Keywords – Electric Vehicle; Eco-Driving; Predictive Cruise Control; LSTM; Model Predictive Control; Drive Cycles; Regenerative Braking; Energy Efficiency.

I. INTRODUCTION

Range anxiety and energy efficiency remain primary adoption barriers for battery electric vehicles (EVs). Connected and automated vehicles can save energy by anticipating the traffic ahead instead of reacting to it [1]: a vehicle that knows a slowdown is coming can coast early instead of braking late. Classical cruise controllers such as PID and bang-bang control react to speed error after it has accumulated [2],[3]. Model predictive control (MPC) plans a torque sequence over a finite horizon [4],[5], but it needs a forecast of the reference. Zhou et al. [6] combined an LSTM [7] predictor of the preceding vehicle's speed with nonlinear MPC for a connected EV, and LSTM networks are established predictors of traffic speed [8]. Learning-based eco-driving and look-ahead energy management have also been studied with MPC and reinforcement learning [9],[10].

What is rarely measured is how much of a predictive controller's benefit comes from the forecast itself. A fair answer needs a vehicle model that can physically follow the test profiles, standard test profiles rather than a hand-made one, and reference forecasts that bracket the learned one, from no forecast to perfect knowledge of the future. Energy must also be compared at equal tracking quality, since a controller that follows the reference loosely saves energy trivially.

This paper makes the following contributions:

- (i) An evaluation of LSTM-enhanced PID and MPC eco-cruise control on the EPA UDDS, HWFET and US06 drive cycles, with a drivetrain model that can follow all three.
- (ii) A forecast bracket of persistence, constant-acceleration extrapolation, the LSTM and an oracle preview, so the LSTM's contribution can be read against what any forecast could add.
- (iii) An equal-tracking energy comparison based on sweeping the MPC energy weight, and a five-seed evaluation of the LSTM.
- (iv) The finding that a perfect 2 s preview is worth 0.7–4.3% energy, of which the LSTM recovers little, because its forecasts are no better than extrapolation and vary from step to step.

Fig. 1 summarizes the setup. Section II presents the vehicle model, the drive cycles and the LSTM forecaster; Section III the forecasts and controllers; Section IV the evaluation protocol; Section V the results. Section VI discusses them and Section VII concludes.

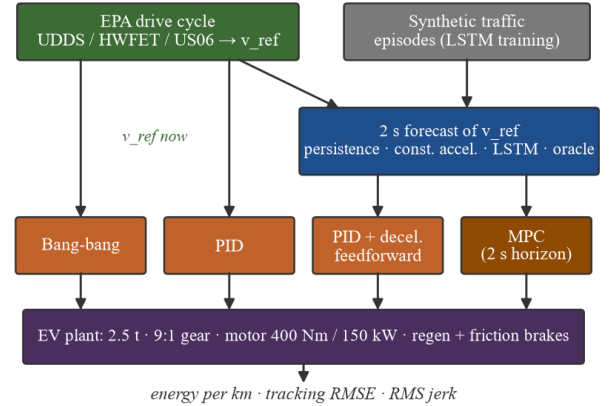


Fig. 1. Evaluation setup: the drive cycle provides the reference speed; predictive controllers also receive one of four 2 s forecasts.

II. SYSTEM MODEL

A. Electric Vehicle Dynamics

The EV is modelled as a point mass with longitudinal dynamics, integrated with $\Delta t = 0.1$ s:

$$m \frac{dv}{dt} = \frac{T_w}{r_w} - \frac{1}{2} \rho C_d A_f v^2 - mg C_r$$

where T_w is the wheel torque, $r_w = 0.36$ m, $m = 2,500$ kg, $\rho = 1.206$ kg/m³, $C_d = 0.28$, $A_f = 2.5$ m² and $C_r = 0.015$ on a flat road. A single-speed reduction gear ($G = 9$) connects the motor to the wheels, so the available drive torque is limited by both motor torque and power [11]:

$$T_w \leq G \cdot \min(T_{m,max}, \frac{P_{m,max}}{\omega_m}), \quad \omega_m = \frac{Gv}{r_w}$$

with $T_{m,max} = 400$ Nm and $P_{m,max} = 150$ kW. Braking uses regeneration first, up to 200 Nm or 60 kW at the motor and only above 1 m/s, and friction brakes for the rest. At 5, 15, 25 and 35 m/s the vehicle can accelerate at 3.85, 3.81, 2.15 and 1.36

m/s², enough for every test cycle, including the 3.8 m/s² peak of US06. Battery power and net energy are

$$P_{batt} = \frac{P_m}{\eta_m} (P_m \geq 0), \quad P_{batt} = \eta_r P_m (P_m < 0)$$

$$E = \frac{1}{3600} \sum_k P_{batt,k} \Delta t$$

with $\eta_m = 0.92$ and $\eta_r = 0.75$, so recovered energy counts as a credit. Energy is reported in Wh per kilometre driven.

B. Drive Cycles

The test profiles are the EPA UDDS, HWFET and US06 dynamometer schedules [12], given at 1 Hz in mph and interpolated to 0.1 s (Table I, Fig. 2). The cycle speed is the reference v_{ref} that the vehicle must follow: the traffic ahead. None of the cycles is used for training or tuning.

TABLE I. EPA DRIVE CYCLES

Cycle	Duration / distance	Max / mean speed (m/s)	Max accel. / decel. (m/s ²)
UDDS (city)	1,369 s / 12.0 km	25.4 / 8.8	1.48 / 1.48
HWFET (highway)	765 s / 16.5 km	26.8 / 21.6	1.43 / 1.48
US06 (aggressive)	600 s / 12.9 km	35.9 / 21.5	3.76 / 3.08

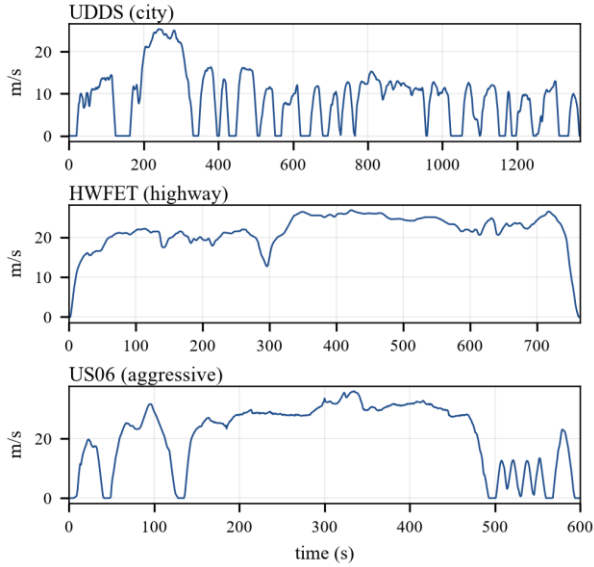


Fig. 2. EPA test cycles used as the reference speed.

C. Synthetic Training Data

The LSTM is trained on synthetic traffic-speed episodes, never on the test cycles. Each 300 s episode at 0.1 s is a sequence of accelerate, cruise, decelerate-or-stop and idle segments, with a slow sinusoidal wobble while cruising, Gaussian smoothing ($\sigma = 0.4$ s) and small sensor noise. Three driving styles (urban, highway and aggressive) differ in cruise speed (4–17, 18–33 and 8–36 m/s), stop probability and acceleration range (up to 3.5 m/s²). Eighty episodes per style are split 80/20 into training and validation by episode, so no validation window overlaps a training window. Windows of 10 s history and 2 s future, cut with a 2.5 s stride, give 22,272 training and 5,568 validation windows.

D. LSTM Speed Forecaster

The forecaster (Fig. 3) keeps the stacked architecture: two LSTM layers with 128 and 64 units, each followed by batch normalization [13] and dropout ($p = 0.2$) [14], and a dense head of 32 ReLU units and 20 outputs. Its two input channels are the 10 s speed history relative to the latest speed, and the latest

speed itself. It predicts the change in speed over the next 20 steps (2 s), which makes "no change" the natural starting point. Training uses Adam [15] with mean-squared error, batch size 128, learning-rate halving on plateau, and early stopping on the validation episodes (patience 5, best weights restored, at most 40 epochs). Training is deterministic for a given seed; five seeds (42, 1, 2, 3, 4) vary both the synthetic data and the initial weights.

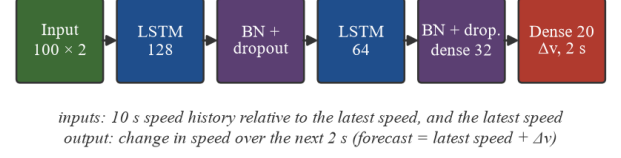


Fig. 3. LSTM speed forecaster.

III. CONTROLLER DESIGNS

A. Forecasts

Every controller sees the current reference speed. The predictive controllers also receive a 2 s forecast $\hat{v}$ of it, from one of four sources: persistence (the current reference repeated, i.e. no forecast); constant acceleration (the last second's acceleration extrapolated, clipped at zero speed); the LSTM; and an oracle that returns the true future reference. The oracle stands for a perfect V2X preview and bounds what any forecaster could add.

B. Bang-Bang Baseline

The baseline applies the full available drive torque when $v_{ref} - v > \delta$ ($\delta = 0.5$ m/s), full braking (at least 1,500 Nm, regeneration first) when $v_{ref} - v < -\delta$, and zero torque inside the band, where the vehicle coasts. It has no look-ahead.

C. PID and LSTM-PID

The PID acts on the speed error $e = v_{ref} - v$ with $k_p = 2,500$ Nm·s/m and $k_i = 150$ Nm/m, the integral clamped to ± 5 m. The predictive variant keeps the deceleration-only feedforward of the original design: when the forecast drops more than 1 m/s below the current reference within 0.8 s, torque is reduced in proportion to the drop:

$$T_w = k_p e + k_i \int e dt - k_{ff} d, \quad d = v_{ref} - \min_{j \leq 8} \hat{v}_j$$

where the last term applies only when $d > 1$ m/s and $k_{ff} = 400$ Nm·s/m. Upcoming accelerations are left to the feedback, since early motoring wastes energy.

D. LSTM-MPC

The MPC minimizes, over the 2 s forecast horizon,

$$J = \sum_{k=1}^{20} [(v_k - \hat{v}_k)^2 + w_e E_k] + w_s \sum_{b=1}^5 (u_b - u_{b-1})^2$$

where v_k is the vehicle speed predicted with the same plant model, E_k the battery energy of step k in mWh (regeneration earns credit through η_r), w_e the energy weight (0.001 by default), $w_s = 2 \times 10^{-8}$ Nm⁻² and u_b the wheel torque of move b . The horizon is split into five blocked moves of 0.4 s, re-optimized every 0.4 s with L-BFGS-B [16] in SciPy [17]. The solver starts from the shifted previous plan or from a proportional-control guess, whichever has the lower cost; the guess matters at standstill, where every braking plan has the same cost. All gains and weights were set on synthetic profiles before any EPA run.

IV. EVALUATION PROTOCOL

Each controller and forecast is simulated on each cycle from $v(0) = v_{ref}(0)$. Three metrics are reported: net battery energy per kilometre, the speed-tracking RMSE between the vehicle speed and the next reference sample, and the RMS jerk. A controller can save energy by following the reference more loosely, so energies are compared at equal tracking: the MPC is swept over

$w_e \in \{0, 0.001, 0.003, 0.01\}$ for each forecast, and the resulting energy–RMSE curve is interpolated, in log RMSE, to the tracking error of the plain PID on the same cycle. The oracle, persistence and constant-acceleration results do not depend on the training seed; LSTM results are reported over five seeds as mean $\pm$ standard deviation.

V. SIMULATION RESULTS

A. Forecast Accuracy

Table II gives the RMSE of the 2 s forecasts. The LSTM halves the error of persistence on every cycle, but constant-acceleration extrapolation is more accurate on the city and highway cycles and only slightly worse on US06. The spread across seeds (0.03–0.05 m/s) is as large as the gap to extrapolation.

TABLE II. RMSE OF THE 2 S FORECAST (m/s)

Cycle	Persistence	Const. accel.	LSTM (5 seeds)
UDDS	0.729	0.292	0.320 ± 0.046
HWFET	0.352	0.133	0.177 ± 0.045
US06	1.133	0.582	0.551 ± 0.034

B. Controllers at Default Settings

Table III lists energy, tracking error and jerk for every controller at its default settings (seed 42). Bang-bang uses 1–5% less energy than PID, but tracks $1.5\text{--}4.5\times$ worse and its jerk is $20\text{--}80\times$ higher: inside its ± 0.5 m/s band it coasts, and coasting is cheap. The deceleration feedforward hardly changes PID energy ($\leq 0.4\%$ over five seeds), even with the oracle forecast, while it slightly tightens tracking. Without a forecast the MPC tracks worse than the PID; with the oracle it tracks best on UDDS and US06 and uses less energy than every controller except bang-bang.

TABLE III. ENERGY (Wh/km) / RMSE (m/s) / RMS JERK (m/s³)

Controller	UDDS	HWFET	US06
Bang-bang	157.2 / 0.354 / 15.9	179.6 / 0.395 / 12.6	237.3 / 0.509 / 15.1
PID	165.1 / 0.213 / 0.39	182.9 / 0.087 / 0.15	240.1 / 0.332 / 0.74
PID + LSTM	165.2 / 0.194 / 0.56	183.0 / 0.084 / 0.22	241.0 / 0.293 / 0.91
PID + oracle	165.2 / 0.189 / 0.47	182.9 / 0.084 / 0.18	239.8 / 0.290 / 0.78
MPC + persist.	163.6 / 0.421 / 1.71	181.7 / 0.181 / 0.46	235.0 / 0.557 / 1.78
MPC + c. accel.	165.1 / 0.179 / 1.80	182.1 / 0.123 / 0.60	243.8 / 0.234 / 2.47
MPC + LSTM	165.5 / 0.078 / 2.02	182.0 / 0.125 / 1.00	247.2 / 0.185 / 3.63
MPC + oracle	163.1 / 0.054 / 0.82	181.4 / 0.127 / 0.46	232.9 / 0.187 / 1.37

C. Energy at Equal Tracking

Fig. 4 shows the energy–tracking trade-off of the MPC for each forecast, and Table IV and Fig. 5 the saving at the PID's tracking error. With the oracle preview the MPC saves 2.6% on UDDS, 0.7% on HWFET and 4.3% on US06. With the LSTM

the saving is $1.0\% \pm 0.8$, $0.0\% \pm 0.6$ and $-0.9\% \pm 1.7$: about 40% of the oracle's saving in the city, none on the highway, and a loss in aggressive driving, where the worst seed costs 3.6%. Constant-acceleration extrapolation gives 0.6%, 0.3% and -0.3% . Without a forecast the MPC never reaches the PID's tracking error at any weight.

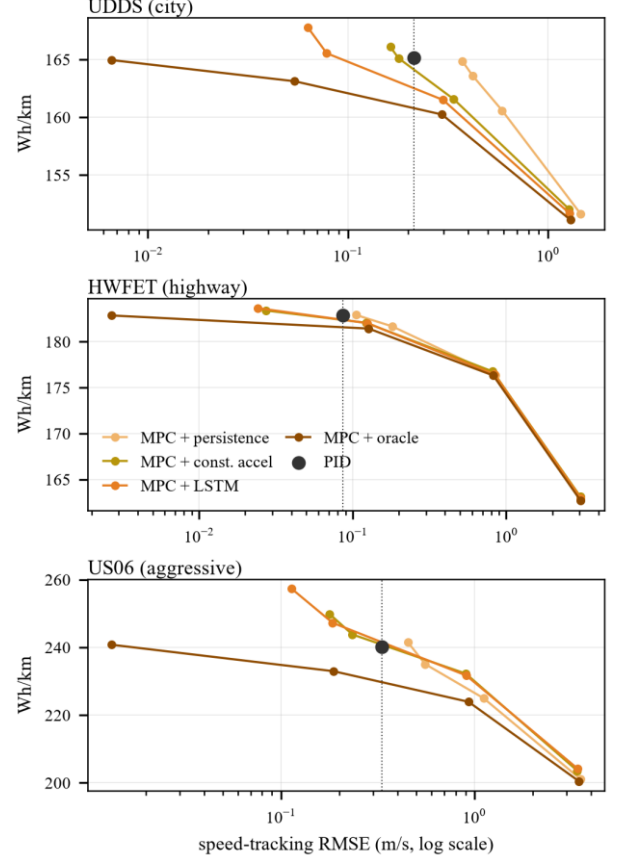


Fig. 4. MPC energy vs tracking error as the energy weight goes from 0 to 0.01, for each forecast; the dotted line marks the PID's tracking error.

TABLE IV. ENERGY SAVING VS PID AT THE PID'S TRACKING ERROR

Cycle	MPC + const. accel.	MPC + LSTM (5 seeds)	MPC + oracle
UDDS	+0.6%	$+1.0\% \pm 0.8$	+2.6%
HWFET	+0.3%	$+0.0\% \pm 0.6$	+0.7%
US06	-0.3%	$-0.9\% \pm 1.7$	+4.3%

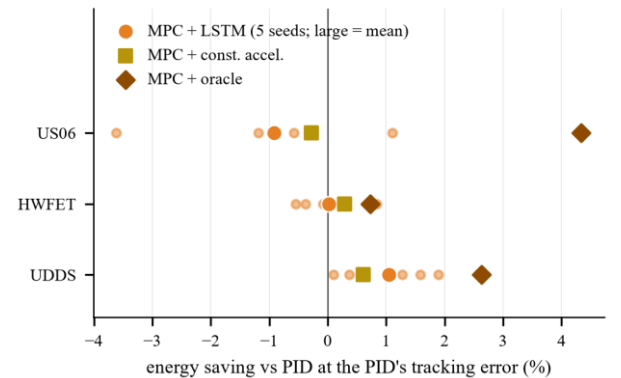


Fig. 5. Energy saving vs PID at the PID's tracking error.

D. Why the LSTM Falls Short

Accuracy alone does not explain the gap: on US06 the LSTM is slightly more accurate than extrapolation but saves less. Its forecast changes noticeably from one step to the next, and the MPC re-plans on each new forecast, so it accelerates and brakes more often. Over five seeds the RMS jerk of MPC + LSTM is 2.3, 1.2 and 3.8 m/s³, against 0.8, 0.5 and 1.4 m/s³ for the oracle MPC (2.5–2.8×), and the extra acceleration costs energy.

VI. DISCUSSION AND LIMITATIONS

Preview information is worth a few percent of energy, most of it in aggressive driving; over 2 s, highway speed barely changes, so there is little to anticipate. A learned forecaster earns its place only if it is both more accurate than simple extrapolation and smooth enough for the controller that consumes it.

Several limitations apply. The LSTM is trained on synthetic traffic only, and real traffic-speed data from V2X or probe vehicles is the natural next step. The vehicle model is longitudinal, with constant motor and regeneration efficiencies and no road grade. The energy differences are a few percent and depend on the plant model, so the comparisons hold the plant fixed and change only the controller or the forecast.

VII. CONCLUSION

This paper evaluated LSTM-enhanced PID and MPC eco-cruise control for an EV on the EPA city, highway and aggressive drive cycles, with forecasts bracketed between no preview and a perfect one, and energy compared at equal tracking. A perfect 2 s preview lets the MPC match the PID's tracking with 0.7–4.3% less energy. The stacked LSTM recovers little of that saving, because its forecasts are no more accurate than constant-acceleration extrapolation and vary from step to step, and the deceleration-only feedforward does not change PID energy. Future work will address:

- (i) training and validation on real traffic-speed data (V2X or probe vehicles);
- (ii) smoothing the forecast, or training the LSTM with a smoothness penalty, to recover more of the oracle's saving;
- (iii) longer preview horizons from V2X, and road-grade preview as in [6];
- (iv) safe car-following constraints and multi-vehicle scenarios.

VIII. REFERENCES

- [1] A. Vahidi and A. Sciarretta, "Energy saving potentials of connected and automated vehicles," *Transp. Res. Part C: Emerg. Technol.*, vol. 95, pp. 822–843, 2018, doi: 10.1016/j.trc.2018.09.001.
- [2] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 33–55, 2016, doi: 10.1109/TIV.2016.2578706.
- [3] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012, doi: 10.1007/978-1-4614-1433-9.
- [4] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.
- [5] D. Q. Mayne, "Model predictive control: Recent developments and future promise," *Automatica*, vol. 50, no. 12, pp. 2967–2986, 2014, doi: 10.1016/j.automatica.2014.10.128.
- [6] Y. Zhou, W. Zhuang, and F. Ju, "Ecological predictive cruise control of connected electric vehicle with predecessor velocity prediction and road grade preview," in *Proc. IEEE Vehicle Power and Propulsion Conf. (VPPC)*, 2021, pp. 1–7, doi: 10.1109/VPPC53923.2021.9699287.
- [7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [8] X. Ma, Z. Tao, Y. Wang, H. Yu, and Y. Wang, "Long short-term memory neural network for traffic speed prediction using remote microwave sensor data," *Transp. Res. Part C: Emerg. Technol.*, vol. 54, pp. 187–197, 2015, doi: 10.1016/j.trc.2015.03.014.
- [9] K. Yeom, "Model predictive control and deep reinforcement learning based energy efficient eco-driving for battery electric vehicles," *Energy Rep.*, vol. 8, pp. 34–42, 2022, doi: 10.1016/j.egyr.2022.10.040.
- [10] T. Liu, X. Hu, S. E. Li, and D. Cao, "Reinforcement learning optimized look-ahead energy management of a parallel hybrid electric vehicle," *IEEE/ASME Trans. Mechatronics*, vol. 22, no. 4, pp. 1497–1507, 2017, doi: 10.1109/TMECH.2017.2707338.
- [11] M. Ehsani, Y. Gao, S. Longo, and K. Ebrahimi, *Modern Electric, Hybrid Electric, and Fuel Cell Vehicles*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2018, doi: 10.1201/9780429504884.
- [12] U.S. Environmental Protection Agency, "Dynamometer drive schedules (UDDS, HWFET, US06)." [Online]. Available: <https://www.epa.gov/vehicle-and-fuel-emissions-testing/dynamometer-drive-schedules>
- [13] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.
- [14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, arXiv:1412.6980.
- [16] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A limited memory algorithm for bound constrained optimization," *SIAM J. Sci. Comput.*, vol. 16, no. 5, pp. 1190–1208, 1995, doi: 10.1137/0916069.
- [17] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.